

Lab2

Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

Theory

Maven is a powerful build automation and dependency management tool used primarily for Java projects, although it can be used for projects in other programming languages as well. It provides a comprehensive set of features to help manage the software development lifecycle and simplify the build process.

Maven Build Lifecycle

Maven has three built-in build lifecycles: clean, default, and site.

- **Clean Lifecycle:** Deletes any build output generated by previous builds.
- `clean`: Deletes any build output generated by previous builds.
- **Default Lifecycle:**
 - `validate`: Validates the project structure and verifies if all necessary information is available.
 - `compile`: Compiles the project's source code.
 - `test`: Runs unit tests against compiled source code.
 - `package`: Packages the compiled code into a distributable format (e.g., JAR, WAR).
 - `install`: Installs the package into the local repository for use as a dependency in other projects (.m2).
 - `deploy`: Deploys the package to a remote repository for sharing with other developers or environments (Push to Nexus).
- **Site Lifecycle:**
 - `site`: Generates project documentation and reports.
 - `site-deploy`: Deploys the generated documentation to a remote web server.

Maven – POM

- Project Object Model (POM) refers to the XML files with all the information regarding project and configuration details. It contains the project description, as well as details regarding the versioning and configuration management of the project.

- The XML file is in the project home directory. Maven searches for the POM in the current directory when any given task needs to be executed.
- Some of the configuration that can be specified in the POM are following – project dependencies, plugins, goals, build profiles, project version, developers, mailing list

Dependency management

Dependency management - this allows project authors to directly specify the versions of artifacts to be used when they are encountered in transitive dependencies or in dependencies where no version has been specified.

Maven Plugin

“Maven” is really just a core framework for a collection of Maven Plugins. In other words, plugins are where much of the real action is performed, plugins are used to: create jar files, create war files, compile code, unit test code, create project documentation, and on and on. Almost any action can think of performing on a project is implemented as a Maven plugin.

How to add Dependencies and Plugins in pom.xml

- The newly created project will have a pom.xml file in its root directory. This file is crucial for Maven configuration.

Adding Dependencies

- Open pom.xml.
- To add a dependency according to requirement , find the dependency information on Maven Repository (e.g., <https://mvnrepository.com/>).

Add the dependency snippet within the <dependencies> tag. For example:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>5.2.3.RELEASE</version>
</dependency>
```

Adding Plugins

```
<plugins>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>3.8.1</version>
    <configuration>
```

```
<source>1.8</source>

<target>1.8</target>

</configuration>

</plugin>

</plugins>
```

Procedure to create Maven Project

➤ Create one folder in desktop with name “mavenProject_batch” AND copy this path

1. Creating a Maven Project using command prompt

1) open command prompt and change the directory as follows

```
Cd C:\Users\Desktop\mavenProject_batch
```

2) Verify JDK installation: Ensure you have a JDK installed (not just a JRE).

- Set JAVA_HOME: C:\Program Files\Java\jdk-xx.
- Verify Maven: Run “mvn -version” in your terminal to confirm it is pointing to the correct JDK path.

3) To create mvn project

```
mvn archetype:generate
```

4) Specify Project Details

- Enter the Group ID (usually your organization or package name)
- Enter the Artifact ID (the name of your project).
- Fill in the Version and Packaging (e.g., jar).
- Click Finish to create the project.

5) Open Java Code (App.java) File inside mavenProject_batch folder and paste the following

```
package com.yit.demo;

public class App {

    public int add(int a, int b) {

        return a + b;

    }

    public static void main(String[] args) {

        App app = new App();
```

```

        int result = app.add(2, 3);

        System.out.println("2 + 3 = " + result);

        System.out.println("Application executed successfully!");
    }
}

```

- 6) Open Java Code (AppTest.java) File inside mavenProject_batch folder and copy the following code

```

package com.yit.demo;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class AppTest {

    @Test
    public void testAdd() {
        App app = new App();
        int result = app.add(2, 3);
        System.out.println("Running test: 2 + 3 = " + result);
        assertEquals(5, result);
    }
}

```

- 7) To build and run this project, follow these steps:

1. Compile the Project

```
mvn compile
```

2. Run the Unit Tests

```
mvn test
```

3. **Package the project into a JAR**

```
mvn package
```

4. **Run the application (using JAR)**

```
java -cp target\exp2-1.0-SNAPSHOT.jar com.yit.demo.App
```

2. Creating a Maven Project using IDEs

Initial Set UP

- 1) Window->preference->java->installed JRE->add->jdk from directory->uncheck the JRE and select JDK
- 2) Window->preference->compiler->compiler compliance level->1.8
- 3) Window->preference->maven->add the maven from program files->select maven

Creating Maven project

- 1) project->create maven project->select use default workspace location->next->catalog->all catalog or maven-central->search for maven-archetype-simple->select org.apache.maven->next->group id and archetype->finish
- 2) You can manually navigate the **project folder** named call **exp2** and open the file pom.xml and copy the below code and paste it then save it.
 - **Open Java Code (App.java) File and paste it (code is similar to previous App.java)**
 - **Open Java Code (AppTest.java) File and copy the following code (code is similar to previous AppTest.java)**
- 3) Now click on the maven project name “exp2” and click on src/main/java then click on com.yit.exp2 package, now right click on App.java click on “Run as” Java application.
- 4) Next again click on the maven project name “exp2” and click on src/test/java then click on com.yit.exp2 package, now right click on AppTest.java click on “Run as” Junit Test.